

文本匹配小案例三：Word2Vec 词向量与语义相似度

一、本节课要解决什么问题

在“小案例二”中，我们已经学会了：

- 词袋模型；
- TF-IDF；
- 余弦相似度；
- 用文本相似度给简历和岗位排序。

TF-IDF 很重要，也很适合入门。它能告诉我们：

哪些词在当前文本中重要，并且不是所有文本里都到处出现。

但是 TF-IDF 有一个明显限制：

它更关注词面是否重合，不太理解词语之间的语义关系。

举个例子：

简历 A：熟悉 Python、SQL、数据分析
岗位 B：要求 Python、SQL、数据挖掘
岗位 C：要求 Java、Spring、后端开发

人能看出来：

- 数据分析和数据挖掘 虽然不是同一个词，但方向比较接近；

- 简历 A 和岗位 B 更像；
- 简历 A 和岗位 C 差得比较远。

但如果只看 TF-IDF，**数据分析** 和 **数据挖掘** 是两个不同词，它们不会天然产生“语义接近”的关系。

所以本节课引入 Word2Vec。

一句话理解：

Word2Vec 是一种把词语变成数字向量的方法，让语义相近的词在向量空间中距离更近。

本节课的目标不是把 Word2Vec 讲成深奥的数学模型，而是让你理解：

1. 什么是词向量；
2. 为什么词可以变成向量；
3. Word2Vec 和 TF-IDF 有什么区别；
4. 如何用 Word2Vec 计算简历和岗位的语义相似度；
5. 后续如何迁移到 PySpark MLlib。

二、先用生活例子理解“向量”

1. 向量可以理解为“坐标”

假设我们用两个维度描述饮品：

甜度
咖啡味

那么：

饮品	甜度	咖啡味
美式咖啡	1	9
拿铁	4	7
奶茶	8	1
橙汁	7	0

你会发现：

- 美式咖啡和拿铁更接近；
- 奶茶和橙汁在“甜度”上比较接近；
- 美式咖啡和橙汁差别比较大。

这就是向量的直观意义：

用一组数字表示一个对象在不同维度上的特征。

Word2Vec 做的事情类似，只不过它不是用“甜度、咖啡味”描述饮品，而是用几十维、上百维数字描述词语。

2. 词向量可以理解为“词语的坐标”

如果 Word2Vec 学得比较好，它可能让这些词更接近：

数据分析、数据挖掘、机器学习、统计分析

也可能让这些词更接近：

Java、Spring、MySQL、后端开发

它的目标是让语义或上下文接近的词，在向量空间里也更接近。

三、Word2Vec 的核心思想

1. 看一个词周围经常出现什么词

Word2Vec 背后的一个朴素思想是：

一个词的意思，可以通过它经常和哪些词一起出现来理解。

比如在大量文本中，你经常看到：

Python 数据分析 SQL 报表

Python 数据挖掘 机器学习

Python Spark 数据处理

那么模型会慢慢发现：

- Python 经常和数据相关词一起出现；
- 数据分析 和 数据挖掘 的上下文有一定相似；
- Spark 和 数据处理 经常一起出现。

生活类比：

如果一个同学经常出现在图书馆、自习室、实验室附近，你可能会判断他学习比较认真。

如果另一个词经常出现在“Python、SQL、报表、统计”附近，模型也会判断它和数据分析方向关系较近。

2. CBOW 和 Skip-gram 的直观理解

Word2Vec 常见有两种训练方式：

模型	直观理解
CBOW	根据上下文猜中间词
Skip-gram	根据中间词猜周围词

例句：

我 熟悉 Python 和 数据分析

CBOW 的思路像这样：

我、熟悉、和、数据分析 → 猜 Python

Skip-gram 的思路像这样：

Python → 猜 熟悉、数据分析

初学阶段不需要推导训练公式，只要记住：

Word2Vec 通过“词和上下文的关系”学习词向量。

3. Word2Vec 和 TF-IDF 的区别

对比项	TF-IDF	Word2Vec
关注点	词是否出现、出现得是否重要	词和词的上下文是否相似
输出	每篇文本一个稀疏向量	每个词一个稠密向量
优点	简单、可解释、适合小数据	能表达一定语义关系
缺点	不理解同义词和近义词	小数据训练效果不稳定，可解释性较弱
在本项目中	基础相似度分	语义增强分

本课程建议：

TF-IDF 必须掌握，Word2Vec 作为语义增强加入。

四、课前准备

1. 安装依赖

如果要运行本节课中的本地 Word2Vec 代码，可以安装：

```
pip install gensim scikit-learn pandas -i https://mirrors.aliyun.com/pypi/simple/ --trusted-host mirrors.aliyun.com
```

如果暂时安装不了 `gensim`，也可以先完成“手工词向量”案例，理解词向量和余弦相似度的基本思想。

2. 建议创建的代码文件

建议新建文件夹：

```
text_matching_demo_03_word2vec
```

本讲义中的代码可以保存为：

```
01_manual_word_vector.py  
02_train_word2vec_gensim.py  
03_word_similarity_demo.py  
04_sentence_vector_average.py  
05_resume_job_word2vec_similarity.py  
06_compare_tfidf_word2vec.py  
07_weighted_semantic_score.py  
08_pyspark_word2vec_preview.py
```

五、案例 1：手工模拟词向量

这个案例不依赖 `gensim`。我们先手工给几个词设置向量，理解“语义接近”和“余弦相似度”的关系。

代码清单： `01_manual_word_vector.py`

```
# 01_manual_word_vector.py
# 目标：不用模型训练，先手工模拟几个词向量，理解词向量的直观意义
```

```
import math
```

```
# 假设我们用 3 个维度描述词语：
# 第 1 维：数据方向
# 第 2 维：后端开发方向
# 第 3 维：运维方向
#
# 注意：这些数字是为了教学演示手工设计的，不是真实训练出来的。
```

```
word_vectors = {
    "数据分析": [0.9, 0.1, 0.0],
    "数据挖掘": [0.85, 0.1, 0.05],
    "机器学习": [0.8, 0.1, 0.1],
    "java": [0.1, 0.9, 0.0],
    "spring": [0.1, 0.85, 0.0],
    "linux": [0.1, 0.0, 0.9],
}
```

```
def cosine_similarity(vector_a, vector_b):
    """
    计算两个向量的余弦相似度。
    结果越接近 1，说明方向越接近。
    """
    # 点积：两个向量相同位置的数字相乘后再求和
    # 如果两个向量方向接近，点积通常会比较大
    dot = sum(a * b for a, b in zip(vector_a, vector_b))

    # 计算 vector_a 的长度
    norm_a = math.sqrt(sum(a * a for a in vector_a))
```

```

# 计算 vector_b 的长度
norm_b = math.sqrt(sum(b * b for b in vector_b))

# 如果某个向量长度为 0, 说明它没有有效方向
# 这时不能做除法, 直接返回 0
if norm_a == 0 or norm_b == 0:
    return 0

# 余弦相似度 = 点积 / 两个向量长度的乘积
return dot / (norm_a * norm_b)

# 比较两个数据方向词语, 它们的向量方向应该比较接近
score_1 = cosine_similarity(word_vectors["数据分析"], word_vectors["数据挖掘"])

# 比较数据方向和后端方向, 它们的向量方向应该差别较大
score_2 = cosine_similarity(word_vectors["数据分析"], word_vectors["java"])

# 比较两个后端开发相关词语, 它们应该比较接近
score_3 = cosine_similarity(word_vectors["java"], word_vectors["spring"])

# 打印三个相似度结果
print("数据分析 vs 数据挖掘: ", round(score_1, 4))
print("数据分析 vs java: ", round(score_2, 4))
print("java vs spring: ", round(score_3, 4))

```

运行结果应该怎么理解

你应该看到:

- 数据分析和 数据挖掘 相似度较高;

- `java` 和 `spring` 相似度较高;
- `数据分析` 和 `java` 相似度较低。

这个案例说明:

如果词向量设计得合理, 语义相近的词在向量空间中方向会更接近。

思考题 1

这个案例中的词向量是真实模型训练出来的吗?

参考答案:

不是。这里是为了帮助理解概念而手工设计的数字。真实 Word2Vec 会根据大量语料中词语的上下文关系自动学习词向量。

六、案例 2: 使用 gensim 训练 Word2Vec

下面开始使用 `gensim` 训练一个小型 Word2Vec 模型。

请注意:

课堂小语料训练出来的 Word2Vec 效果不一定稳定, 它主要用于理解流程。真实项目中通常需要更大的语料, 或者使用预训练词向量。

代码清单: `02_train_word2vec_gensim.py`

```
# 02_train_word2vec_gensim.py
# 目标: 使用 gensim 训练一个小型 Word2Vec 模型

from gensim.models import Word2Vec

# 每个内部列表表示一段已经分词后的文本
# 这些文本可以理解为简历或岗位描述中的关键词序列
# Word2Vec 训练时需要的输入格式就是“词列表的列表”
sentences = [
    ["python", "sql", "数据分析", "报表", "可视化"],
    ["python", "数据挖掘", "机器学习", "模型", "预测"],
    ["python", "spark", "hadoop", "数据处理", "大数据"],
    ["java", "spring", "mysql", "后端开发", "接口"],
    ["linux", "shell", "服务器", "部署", "运维"],
    ["excel", "数据分析", "统计分析", "报表", "可视化"],
    ["spark", "pyspark", "数据清洗", "数据处理", "hadoop"],
    ["java", "springboot", "接口开发", "mysql", "后端服务"],
]

# vector_size 表示词向量维度
# window 表示看当前词左右多远范围内的上下文
# min_count=1 表示只出现 1 次的词也参与训练, 适合课堂小语料
# workers=1 和 seed=42 是为了让课堂运行结果尽量稳定
model = Word2Vec(
    # 训练语料: 已经分词后的多段文本
    sentences=sentences,
    # 每个词向量的维度, 这里设置为 20, 方便课堂快速训练
    vector_size=20,
    # 当前词左右各看 3 个词作为上下文
    window=3,
    # 小语料中很多词只出现 1 次, 所以设置 min_count=1
    min_count=1,
```

```
# 使用 1 个线程，结果更稳定，便于课堂复现
workers=1,
# 固定随机种子，让每次训练结果尽量接近
seed=42
)

# 查看某个词的向量
# model.wv 表示训练好的词向量表
# model.wv["python"] 表示取出 python 这个词对应的向量
python_vector = model.wv["python"]

# 打印向量长度，确认它等于 vector_size
print("python 的词向量维度：", len(python_vector))
print("python 的前 5 个向量值：")

# 词向量通常有很多维，这里只打印前 5 个数字，方便观察
print(python_vector[:5])

# 保存模型，后续可以直接加载使用
# 保存后，后面的案例就不需要每次重新训练
model.save("word2vec_resume_job.model")
print("模型已保存：word2vec_resume_job.model")
```

关键参数解释

参数	含义
<code>vector_size</code>	每个词向量有多少个数字
<code>window</code>	训练时观察当前词左右多少个词
<code>min_count</code>	低于多少次出现频率的词会被忽略
<code>workers</code>	训练线程数
<code>seed</code>	随机种子，便于课堂结果稳定

思考题 2

为什么课堂案例中设置 `min_count=1` ？

参考答案：

因为课堂语料很小，很多词可能只出现一次。如果使用较高的 `min_count`，这些词会被模型忽略，学生可能查不到词向量。真实大语料中可以适当提高 `min_count`，过滤低频噪声词。

深度解析：训练模型和推理到底在做什么

很多同学第一次接触模型时，会把“训练”和“使用”混在一起。

其实它们是两个阶段：

训练阶段：让模型从语料中学习词向量

推理阶段：使用已经训练好的词向量完成查询、相似度计算、匹配排序

1. Word2Vec 训练阶段在学什么

训练阶段的输入是很多已经分好词的句子：

```
# sentences 是 Word2Vec 的训练语料
# 每个内部列表是一段已经分好词的文本
sentences = [
    ["python", "sql", "数据分析", "报表", "可视化"],
    ["python", "数据挖掘", "机器学习", "模型", "预测"],
    ["java", "spring", "mysql", "后端开发", "接口"],
]
```

Word2Vec 会观察词和词经常一起出现的关系，然后逐步调整每个词的向量，让“上下文相似”的词在向量空间里更接近。

可以这样理解：

训练就是让模型不断移动词语在地图上的位置，让经常出现在相似语境中的词靠得更近。

注意：Word2Vec 并不是“理解”这些词的中文意思或英文意思，而是根据大量上下文统计规律学出向量。向量的形成过程可以简化为三步。

第一步：先给每个词随机分配一个向量。

刚开始模型并不知道 `python`、`sql`、`java` 分别是什么意思，所以它只能先随机给每个词一个位置：

```
python -> 随机向量
sql    -> 随机向量
java   -> 随机向量
```

这些初始向量没有明显规律，可能完全不符合人的直觉。

第二步：从语料中制造“上下文预测任务”。

假设有一句分好词的文本：

python sql 数据分析 报表 可视化

如果当前词是 数据分析，窗口大小 window=2，那么它附近的上下文词可能是：

python、sql、报表、可视化

Word2Vec 会利用这样的上下文关系训练模型。前面已经讲过，CBOW 是“根据周围词猜中间词”，Skip-gram 是“根据中间词猜周围词”。例如 Skip-gram 会做类似这样的任务：

看到 数据分析，尽量预测出 sql、报表、可视化 等上下文词。

第三步：预测错了，就调整向量。

如果模型看到 数据分析，却预测不出 报表、可视化，说明当前向量位置还不合理。

训练程序会根据误差不断调整向量：

让 数据分析 更容易预测到 报表、可视化
让 java 不那么容易预测到 报表、可视化

经过很多轮训练后，经常出现在相似上下文里的词，会被调整到相近的位置。

所以最终可能形成这样的关系：

数据分析、报表、可视化	靠得更近
java、spring、后端开发	靠得更近
spark、hadoop、数据处理	靠得更近

训练结束时，模型保存的是一张“词向量表”：

```
python -> 一串数字  
sql     -> 一串数字  
java    -> 一串数字
```

后面再做相似词查询、简历岗位匹配时，就不需要重新训练，而是直接使用这张词向量表。

一句话总结：

Word2Vec 的词向量不是人工规定的，而是模型通过大量“根据上下文预测词”的练习，一点点调整出来的。

2. 训练出来的模型里保存了什么

执行：

```
# 把训练好的 Word2Vec 模型保存到文件  
model.save("word2vec_resume_job.model")
```

保存下来的模型主要包含：

- 词表：模型认识哪些词；
- 每个词对应的向量；
- 模型参数：比如向量维度、窗口大小等；
- 继续训练或加载使用需要的内部信息。

其中最重要的是：

词 -> 向量

例如可以想象成：

```
python -> [0.12, -0.03, 0.44, ...]
sql    -> [0.10, -0.02, 0.39, ...]
java   -> [-0.31, 0.22, 0.08, ...]
```

这些数字本身不好直接解释，但它们的方向和距离可以用来计算相似度。

3. 推理阶段在做什么

推理阶段不再重新学习词向量，而是直接使用已经训练好的模型。

例如：

```
# 从文件中加载已经训练好的 Word2Vec 模型
model = Word2Vec.load("word2vec_resume_job.model")
```

加载模型后，可以做几类事情。

第一类：查某个词的向量。

```
# 从词向量表中取出 python 对应的向量
vector = model.wv["python"]
```

意思是：

从模型保存的词向量表里，取出 `python` 这个词的向量。

第二类：查相似词。

```
# 查找与“数据分析”最相似的前 5 个词
model.wv.most_similar("数据分析", topn=5)
```

意思是：

找出向量空间中离 数据分析 比较近的 5 个词。

第三类：计算两个词的相似度。

```
# 计算“数据分析”和“数据挖掘”两个词向量之间的相似度
model.wv.similarity("数据分析", "数据挖掘")
```

意思是：

计算两个词向量的余弦相似度。

第四类：把一段文本转换成句向量。

例如一份简历：

```
# 一份简历分词后的结果示例
["python", "sql", "数据分析", "可视化"]
```

可以先取出每个词的向量，再求平均，得到整段简历的向量。

后面案例中的：

```
# 调用函数，把词列表转换成句向量
get_sentence_vector(tokens, model)
```

就是在做这件事。

4. 训练和推理的区别

对比项	训练阶段	推理阶段
目标	学出词向量	使用词向量解决任务
输入	大量分词文本	新的简历、岗位或查询词
输出	模型文件、词向量表	相似词、相似度、匹配排序
是否改变模型	会改变模型参数	通常不改变模型参数
速度	相对较慢	通常较快

一句话总结：

训练是“学习地图”，推理是“拿着地图查位置、算距离、做推荐”。

5. 为什么课堂小模型效果可能不稳定

课堂案例只有几条句子，语料太小。

这会导致：

- 有些词只出现 1 次，模型很难学准；
- 上下文太少，词和词的关系不稳定；
- 每次训练结果可能有轻微变化；
- 相似词结果可能不完全符合人的直觉。

所以本节课训练的小模型主要用于理解流程。

真实项目中更常见的做法是：

更多简历和岗位语料
或
使用已经训练好的预训练词向量
或
结合 TF-IDF、多维评分一起使用

这也是为什么后面我们不会只依赖 Word2Vec，而是会把它作为综合匹配分中的一个语义特征。

七、案例 3：查看词语相似度

训练好 Word2Vec 后，可以查看哪些词和某个词更相近。

代码清单： `03_word_similarity_demo.py`

```
# 03_word_similarity_demo.py
# 目标: 查看 Word2Vec 模型认为哪些词更相似

from gensim.models import Word2Vec

# 加载前一个案例训练并保存的 Word2Vec 模型
model = Word2Vec.load("word2vec_resume_job.model")

# most_similar 会返回与指定词最相似的若干个词
# topn=5 表示返回最相似的前 5 个词
similar_words = model.wv.most_similar("数据分析", topn=5)

print("与 数据分析 最相似的词: ")
for word, score in similar_words:
    # score 是 Word2Vec 计算出的词向量余弦相似度
    print(word, "=>", round(score, 4))

print("\n两个词之间的相似度: ")
# similarity 可以直接计算两个词向量之间的相似度
print("数据分析 vs 数据挖掘: ", round(model.wv.similarity("数据分析", "数据挖掘"), 4))
print("数据分析 vs java: ", round(model.wv.similarity("数据分析", "java"), 4))
```

运行结果如何看

如果某个词排在前面, 说明模型认为它和查询词在向量空间中比较接近。

但要注意:

小语料训练结果可能不完全符合人的直觉。

如果你发现结果不理想, 不一定是代码错了。更可能是因为语料太少。

讨论题 1

如果 Word2Vec 训练结果和人的直觉不一致，可能是什么原因？

参考答案：

可能是训练语料太少、词出现次数太低、上下文不够丰富、分词结果不好，或者参数设置不合适。Word2Vec 需要通过大量上下文学习词义，小样本很难训练出稳定语义。

八、案例 4：把一段文本变成句向量

Word2Vec 给每个词一个向量。

但简历和岗位通常是一段文本，包含多个词。

我们需要把多个词向量合成一个文本向量。

一种简单方法是：

对文本中所有词向量求平均。

例如：

简历：python sql 数据分析

可以理解为：

简历向量 = (python向量 + sql向量 + 数据分析向量) / 3

这种方法简单、直观，适合入门项目。

代码清单： `04_sentence_vector_average.py`

```
# 04_sentence_vector_average.py
# 目标：用平均词向量表示一段文本

import numpy as np
from gensim.models import Word2Vec

# 加载已经训练好的 Word2Vec 模型
model = Word2Vec.load("word2vec_resume_job.model")

def get_sentence_vector(tokens, model):
    """
    把分词后的文本转换成句向量。

    参数：
        tokens: 分词后的词列表，例如 ["python", "sql", "数据分析"]
        model: 训练好的 Word2Vec 模型

    返回：
        一个 numpy 数组，表示整段文本的向量
    """
    # vectors 用来保存文本中每个词对应的词向量
    vectors = []

    for word in tokens:
        # 只有模型词表中存在的词，才能取到词向量
        # 如果词不在模型词表中，直接跳过，避免报错
        if word in model.wv:
            vectors.append(model.wv[word])

    # 如果文本中没有任何词能找到向量，则返回全 0 向量
    if len(vectors) == 0:
```

```
        return np.zeros(model.vector_size)

    # axis=0 表示按列求平均，得到一个向量
    # 例如有 4 个词，每个词 20 维，平均后仍然是 20 维
    return np.mean(vectors, axis=0)

# 一段简历文本的分词结果
resume_tokens = ["python", "sql", "数据分析", "可视化"]

# 把多个词向量求平均，得到整段简历的句向量
resume_vector = get_sentence_vector(resume_tokens, model)

print("句向量维度：", len(resume_vector))
print("句向量前 5 个值：")
print(resume_vector[:5])
```

思考题 3

为什么要处理“词不在模型词表中”的情况？

参考答案：

因为 Word2Vec 只能为训练时见过的词生成向量。如果新文本中出现模型没有见过的词，直接取向量会报错。实际项目中需要跳过未知词，或者用默认向量处理。

九、案例 5：用 Word2Vec 计算简历岗位相似度

现在我们把 Word2Vec 用到简历匹配中。

流程是：

```
简历分词
↓
简历句向量
↓
岗位分词
↓
岗位句向量
↓
使用 sklearn 计算余弦相似度
```

注意：小案例二已经手工讲过余弦相似度公式，所以从这个案例开始，我们不再重复手写公式，而是直接使用 `scikit-learn` 提供的 `cosine_similarity`。

代码清单： `05_resume_job_word2vec_similarity.py`

```
# 05_resume_job_word2vec_similarity.py
# 目标: 使用 Word2Vec 计算一份简历和多个岗位的语义相似度

import numpy as np
from gensim.models import Word2Vec
from sklearn.metrics.pairwise import cosine_similarity

# 加载前面训练好的 Word2Vec 模型
# 这个模型已经学到了每个词对应的向量
model = Word2Vec.load("word2vec_resume_job.model")

def get_sentence_vector(tokens, model):
    """
    把一段文本的词列表转换成句向量。

    做法:
    1. 找到每个词的词向量;
    2. 把多个词向量求平均;
    3. 用平均向量代表整段文本。
    """
    # 保存当前文本中所有能找到的词向量
    vectors = []

    for word in tokens:
        # 只有词在模型词表中时, 才能取出词向量
        # 如果不判断, 遇到未知词会报错
        if word in model.wv:
            vectors.append(model.wv[word])

    # 如果一个词都没有找到, 返回全 0 向量
    # 这样后续计算相似度时不会报错
```

```
if len(vectors) == 0:
    return np.zeros(model.vector_size)

# 对所有词向量按维度求平均
# 结果仍然是一个 model.vector_size 维的向量
return np.mean(vectors, axis=0)

# 一份学生简历
# tokens 是已经分词、清洗、标准化后的关键词列表
resume = {
    "name": "张同学",
    "tokens": ["python", "sql", "数据分析", "可视化", "报表"]
}

# 多个岗位
# 每个岗位也用 tokens 保存关键词列表
jobs = [
    {
        "job_name": "数据分析实习生",
        "tokens": ["python", "sql", "数据分析", "报表", "excel"]
    },
    {
        "job_name": "大数据开发实习生",
        "tokens": ["python", "spark", "hadoop", "数据处理"]
    },
    {
        "job_name": "后端开发实习生",
        "tokens": ["java", "spring", "mysql", "后端开发"]
    },
]
```

```

# 先把简历转换成句向量
resume_vector = get_sentence_vector(resume["tokens"], model)

# results 用来保存每个岗位的匹配结果
results = []

for job in jobs:
    # 把当前岗位转换成句向量
    job_vector = get_sentence_vector(job["tokens"], model)

    # 使用 scikit-learn 计算简历句向量和岗位句向量的余弦相似度
    # cosine_similarity 要求输入是二维数组:
    # [resume_vector] 表示“只有一条简历向量”
    # [job_vector] 表示“只有一个岗位向量”
    # 返回结果形如 [[0.83]], 所以用 [0][0] 取出里面的分数
    score = cosine_similarity([resume_vector], [job_vector])[0][0]

    results.append({
        "job_name": job["job_name"],
        # 乘以 100 是为了把 0~1 的相似度转换成百分制分数
        "word2vec_score": score * 100
    })

# 按 Word2Vec 相似度从高到低排序
results = sorted(results, key=lambda item: item["word2vec_score"], reverse=True)

print("Word2Vec 岗位相似度排序: ")
for item in results:
    print(item["job_name"], "=>", round(item["word2vec_score"], 2))

```

运行结果解读

如果语料和训练结果比较合理，数据分析相关岗位应该更靠前。

如果排序不符合直觉，优先回到前面的“训练与推理深度解析”检查语料规模、分词质量和上下文是否足够。

十、案例 6：对比 TF-IDF 和 Word2Vec

TF-IDF 和 Word2Vec 都能算文本相似度，但它们关注点不同。

下面做一个对比案例。

代码清单： `06_compare_tfidf_word2vec.py`

```
# 06_compare_tfidf_word2vec.py
# 目标: 对比 TF-IDF 相似度和 Word2Vec 相似度

import numpy as np
from gensim.models import Word2Vec
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# 加载已经训练好的 Word2Vec 模型
model = Word2Vec.load("word2vec_resume_job.model")

def get_sentence_vector(tokens, model):
    """
    把词列表转换成平均词向量。
    """
    # 保存当前文本中出现、并且模型认识的词向量
    vectors = []

    for word in tokens:
        if word in model.wv:
            vectors.append(model.wv[word])

    # 如果没有任何词能找到向量, 返回全 0 向量
    if len(vectors) == 0:
        return np.zeros(model.vector_size)

    # 对词向量求平均, 得到句向量
    return np.mean(vectors, axis=0)

# 一份简历的关键词列表
```

```
resume_tokens = ["python", "sql", "数据分析", "可视化"]

# 三个岗位的关键词列表
jobs = [
    {
        "job_name": "数据分析实习生",
        "tokens": ["python", "sql", "数据分析", "报表"]
    },
    {
        "job_name": "数据挖掘实习生",
        "tokens": ["python", "数据挖掘", "机器学习", "模型"]
    },
    {
        "job_name": "后端开发实习生",
        "tokens": ["java", "spring", "mysql", "后端开发"]
    },
]

# 第一步: 计算 TF-IDF 相似度
# TfidfVectorizer 需要输入字符串, 所以这里把词列表用空格拼接
documents = [" ".join(resume_tokens)] + [" ".join(job["tokens"]) for job in jobs]

# 创建 TF-IDF 向量化工具
tfidf = TfidfVectorizer()

# 同时对简历和岗位建立统一词表, 并转换成 TF-IDF 矩阵
tfidf_matrix = tfidf.fit_transform(documents)

# 第 0 行是简历向量
resume_tfidf = tfidf_matrix[0]

# 第 1 行到最后是岗位向量
```

```

job_tfidf = tfidf_matrix[1:]

# 使用 scikit-learn 计算简历和所有岗位的余弦相似度
tfidf_scores = cosine_similarity(resume_tfidf, job_tfidf)[0]

# 第二步：计算 Word2Vec 相似度
# 先把简历关键词转换成 Word2Vec 句向量
resume_w2v = get_sentence_vector(resume_tokens, model)

# rows 用来保存对比结果
rows = []

for index, job in enumerate(jobs):
    # 当前岗位的 Word2Vec 句向量
    job_w2v = get_sentence_vector(job["tokens"], model)

    # 当前岗位的 Word2Vec 相似度
    # 这里同样使用 scikit-learn 的 cosine_similarity
    # 因为前面小案例二已经讲过余弦相似度公式，小案例三重点放在 Word2Vec 向量
    word2vec_score = cosine_similarity([resume_w2v], [job_w2v])[0][0]

    rows.append({
        "job_name": job["job_name"],
        # TF-IDF 分数来自 tfidf_scores, 对应同一个岗位下标 index
        "tfidf_score": tfidf_scores[index] * 100,
        # Word2Vec 分数是刚刚用句向量计算出来的
        "word2vec_score": word2vec_score * 100,
    })

print("TF-IDF 与 Word2Vec 对比: ")
for row in rows:
    print(

```

```
row["job_name"],  
"TF-IDF:", round(row["tfidf_score"], 2),  
"Word2Vec:", round(row["word2vec_score"], 2)  
)
```

讨论题 2

如果某个岗位和简历共同词不多，但 Word2Vec 分数较高，说明什么？

参考答案：

可能说明两段文本虽然词面重合较少，但词语语义或上下文方向接近。例如“数据分析”和“数据挖掘”不是完全相同的词，但在数据岗位语境中可能比较接近。

十一、案例 7：合成综合语义分

在项目中，我们可以同时保留 TF-IDF 分和 Word2Vec 分。

一个简单做法是：

```
综合语义分 = TF-IDF分 × 0.6 + Word2Vec分 × 0.4
```

为什么 TF-IDF 权重可以稍高？

因为在小型教学数据集中：

- TF-IDF 更稳定；
- TF-IDF 更容易解释；

- Word2Vec 语义更强，但小语料下可能波动。

代码清单： `07_weighted_semantic_score.py`

```
# 07_weighted_semantic_score.py
# 目标: 把 TF-IDF 相似度和 Word2Vec 相似度合成为综合语义分
```

```
# 这里假设前面已经分别算好了每个岗位的两个分数:
# tfidf_score: 关键词重合和词频区分度带来的相似度
# word2vec_score: 词向量语义接近程度带来的相似度
```

```
jobs = [
    {
        "job_name": "数据分析实习生",
        "tfidf_score": 78,
        "word2vec_score": 82,
    },
    {
        "job_name": "数据挖掘实习生",
        "tfidf_score": 45,
        "word2vec_score": 76,
    },
    {
        "job_name": "后端开发实习生",
        "tfidf_score": 10,
        "word2vec_score": 18,
    },
]
```

```
def calculate_semantic_score(tfidf_score, word2vec_score):
    """
    计算综合语义分。

    这里让 TF-IDF 占 60%, Word2Vec 占 40%。
    你也可以根据项目情况调整权重。
    """
```

```
# TF-IDF 更可解释, 能说明哪些关键词真正重合
# Word2Vec 更偏语义, 能捕捉一些词面不完全相同但意思接近的情况
# 这里用加权求和, 把两个分数融合成一个综合分
return tfidf_score * 0.6 + word2vec_score * 0.4

for job in jobs:
    # 对当前岗位计算综合语义分
    semantic_score = calculate_semantic_score(
        job["tfidf_score"],
        job["word2vec_score"]
    )

    # 把计算结果写回当前岗位字典
    job["semantic_score"] = semantic_score

# 按综合语义分从高到低排序
jobs = sorted(jobs, key=lambda item: item["semantic_score"], reverse=True)

print("综合语义分排序: ")
for job in jobs:
    print(
        job["job_name"],
        "TF-IDF:", job["tfidf_score"],
        "Word2Vec:", job["word2vec_score"],
        "综合语义分:", round(job["semantic_score"], 2)
    )
```

思考题 4

为什么不直接用 Word2Vec 分数替代 TF-IDF 分数?

参考答案：

因为 TF-IDF 简单、稳定、可解释，特别适合小型教学数据；Word2Vec 能补充语义信息，但小语料训练效果可能不稳定。二者结合更适合作为教学项目方案。

十二、案例 8：PySpark MLlib Word2Vec 预览

后续项目会迁移到 PySpark。Spark MLlib 中也提供了 Word2Vec。

下面代码是一个预览案例，适合在已经配置好 PySpark 的环境中运行。

代码清单： `08_pyspark_word2vec_preview.py`

```
# 08_pyspark_word2vec_preview.py
# 目标: 预览如何在 PySpark 中使用 Word2Vec

from pyspark.sql import SparkSession
from pyspark.ml.feature import Word2Vec

# 创建 SparkSession
# local[*] 表示在本机使用所有可用 CPU 核心运行
spark = SparkSession.builder \
    .appName("Word2VecPreview") \
    .master("local[*]") \
    .getOrCreate()

# words 列必须是数组形式, 里面是已经分好的词
# 每一行代表一段文本
# 第 0 列是 id, 第 1 列是分词后的词列表
data = [
    (0, ["python", "sql", "数据分析", "可视化"]),
    (1, ["python", "数据挖掘", "机器学习", "模型"]),
    (2, ["java", "spring", "mysql", "后端开发"]),
]

# 把 Python 列表转换成 Spark DataFrame
# 列名分别是 id 和 words
df = spark.createDataFrame(data, ["id", "words"])

# 创建 PySpark 版 Word2Vec 工具
word2vec = Word2Vec(
    # 词向量维度
    vectorSize=20,
    # 小数据演示时, 出现 1 次的词也参与训练
    minCount=1,
```

```
# 输入列：必须是数组类型，也就是已经分好的词列表
inputCol="words",
# 输出列：每一行文本对应的 Word2Vec 句向量
outputCol="word2vec_features"
)

# 在 Spark DataFrame 上训练 Word2Vec 模型
model = word2vec.fit(df)

# 使用模型把 words 列转换成 word2vec_features 向量列
result = model.transform(df)

# 展示结果
# truncate=False 表示不要截断太长的向量内容
result.select("id", "words", "word2vec_features").show(truncate=False)

# 关闭 SparkSession, 释放资源
spark.stop()
```

代码说明

代码	含义
<code>inputCol="words"</code>	输入列，必须是分词后的数组
<code>outputCol="word2vec_features"</code>	输出列，保存文本向量
<code>vectorSize=20</code>	词向量维度
<code>minCount=1</code>	小语料中保留低频词

讨论题 3

PySpark 版 Word2Vec 和本地 gensim 版 Word2Vec 有什么共同点？

参考答案：

它们都把分词后的文本作为输入，都可以训练词向量，并把文本转换成向量。不同点是 PySpark 更适合处理大规模分布式数据，本地 gensim 更适合入门演示和小规模实验。

十三、如何接入后续多维评分

本节课结束后，我们可以得到：

```
tfidf_score  
word2vec_score  
semantic_score
```

后续“小案例四：多维评分与可解释匹配”中，可以这样使用：

$$\text{综合语义分} = \text{TF-IDF分} \times 0.6 + \text{Word2Vec分} \times 0.4$$
$$\begin{aligned} \text{综合匹配分} = & \\ & \text{技能分} \times 0.40 \\ & + \text{综合语义分} \times 0.30 \\ & + \text{学历分} \times 0.15 \\ & + \text{经验分} \times 0.15 \end{aligned}$$

也可以不合成 `semantic_score`，直接把 `TF-IDF` 分 和 `Word2Vec` 分 分别放进总分公式。教学项目中更推荐先合成综合语义分，因为结构更清楚，后续解释也更方便。

十四、常见问题

1. Word2Vec 能真正理解句子意思吗

不能完全理解。

Word2Vec 主要学习词语的上下文关系，它比简单关键词匹配更进一步，但还不能像人一样理解完整句子逻辑。训练与推理的细节已经在案例 2 后集中讲过，这里只记住它的边界。

例如：

```
熟悉 Python  
不熟悉 Python
```

这两句话都包含 `Python`。Word2Vec 本身也不一定能很好理解否定关系。

2. 为什么小语料训练效果不好

因为 Word2Vec 依赖大量上下文学习词语关系。小语料中词出现次数少、上下文不丰富，所以结果容易波动。本课程把 Word2Vec 定位为语义增强和迁移准备，而不是唯一核心算法。

3. 是否可以使用预训练词向量

可以，但两周实训中不建议作为核心要求。

预训练词向量文件通常较大，下载和配置可能占用课堂时间。可以作为优秀小组的拓展方向。

4. Word2Vec 和 BERT 有什么区别

简单理解：

- Word2Vec 主要给每个词学习一个固定向量；
- BERT 会根据上下文动态理解词语含义，能力更强；
- BERT 更复杂，对环境和算力要求更高。

本课程先学习 Word2Vec，是为了从 TF-IDF 平稳过渡到更高级的语义表示。

十五、课堂练习

练习 1：解释词向量

请用自己的话解释：

什么是词向量？

参考答案：

词向量就是用一组数字表示一个词。语义或上下文相近的词，它们的向量方向通常更接近。

练习 2：对比 TF-IDF 和 Word2Vec

问题：

TF-IDF 和 Word2Vec 的核心区别是什么？

参考答案：

TF-IDF 主要看词是否出现以及词的重要程度，更关注词面重合；Word2Vec 通过上下文学习词向量，更关注词语之间的语义接近关系。

练习 3：计算综合语义分

已知：

TF-IDF 分：70
Word2Vec 分：85

使用公式：

综合语义分 = TF-IDF分 $\times$ 0.6 + Word2Vec分 $\times$ 0.4

综合语义分是多少？

参考答案：

$70 \times 0.6 = 42$
 $85 \times 0.4 = 34$
综合语义分 = 76

练习 4：判断 Word2Vec 是否适合单独使用

如果你的项目只有 10 份简历和 8 个岗位，是否建议完全依赖自己训练的 Word2Vec？

参考答案：

不建议。因为语料太少，Word2Vec 很难学到稳定语义。更合理的做法是保留 TF-IDF 作为基础分数，再把 Word2Vec 作为语义增强或对比实验。

十六、本节课小结

本节课我们完成了从“关键词相似”到“语义向量相似”的过渡。

你需要掌握：

知识点	你应该能说清楚什么
词向量	用一组数字表示词语
Word2Vec	根据上下文学习词向量
CBOW	根据周围词猜中间词
Skip-gram	根据中间词猜周围词
句向量	用平均词向量表示一段文本
Word2Vec 相似度	用句向量余弦相似度比较简历和岗位
TF-IDF 对比	TF-IDF 稳定可解释，Word2Vec 提供语义增强
PySpark 迁移	Spark MLlib 提供 Word2Vec，可用于大数据版项目

请记住本节课最重要的一句话：

TF-IDF 告诉我们“哪些词重要”，Word2Vec 尝试告诉我们“哪些词语义接近”。

下一节课会继续把 `tfidf_score`、`word2vec_score`、技能分、学历分、经验分组合起来，形成可解释的综合匹配分。